

moodle.jku.at

Exam: Überprüfung des Testversuchs

6–8 Minuten

Begonnen am	Monday, 26. June 2023, 13:57
Status	Beendet
Beendet am	Monday, 26. June 2023, 14:12
Verbrauchte Zeit	15 Minuten
Bewertung	5,50 von 10,00 (55%)

Frage 1

Vollständig

Erreichte Punkte 1,00 von 1,00

Fragetext

What is the best possible description for the output of the following snippet of code?

```
@torch.enable_grad()
def compute_loss(network: nn.Module, loss_func:
nn.Module, x: torch.Tensor, y: torch.Tensor):
    logits = network(x)
    err = loss_func(logits, y)
```

```
err.backward()  
return err.item()
```

Frage 2

Vollständig

Erreichte Punkte 1,00 von 1,00

Frage text

What is the most accurate description for the following snippet of code?

```
net = nn.Sequential()  
net.add_module("features", FeatureNetwork(...))  
net.add_module("classifier", nn.Linear(...))  
  
state_dict = torch.load(...)  
net.features.load_state_dict(state_dict)  
nn.init.kaiming_uniform_(net.classifier.weight)  
nn.init.zeros_(net.classifier.bias)  
  
net.features.requires_grad_(True)  
net.classifier.requires_grad_(True)
```

Frage 3

Vollständig

Erreichte Punkte 1,00 von 2,00

Frage text

Which *two* of the following concepts do **not** belong in the list of answer options? Concepts are said to belong together if they fit within one of the assignments you made this semester.

Frage 4

Vollständig

Erreichte Punkte 1,00 von 1,00

Frage text

Which of the following lines of code implements the *ascent* of projected gradient ascent, which is used to compute adversarial examples?

Frage 5

Vollständig

Erreichte Punkte 1,00 von 1,00

Frage text

The code snippet below aims to produce the **objective to be minimised** for the generator of a GAN. You can assume the label for real (fake) labels to be one (zero). What is wrong with this code?

```
def generator_error(gan: nn.ModuleDict, x:
torch.Tensor):
    bce = nn.BCEWithLogitsLoss()
    z = torch.randn_like(x)
    with torch.enable_grad():
```

```
fakes = gan.generator(z)
logits = gan.discriminator(fakes)
real_labels = torch.ones_like(logits)
err = bce(logits, real_labels)
return -err
```

Frage 6

Vollständig

Erreichte Punkte 0,50 von 1,00

Frage text

How would you implement the forward pass of an additive coupling layer?

Frage 7

Vollständig

Erreichte Punkte 0,00 von 3,00

Frage text

Match the code snippets to the corresponding ML research papers.

Hint: Try to find algorithms, figures or equations in the papers, there should be no need to read them.

Click on the selected ML research paper to view it:

- **Going Deeper with Convolutions**
- **Rethinking the Inception Architecture for Computer Vision**

► Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning

```
import torch
from torch import nn

class InceptionBlock(nn.Module):

    def __init__(self, in_channels,
                  o1x1_channels, o3x3_channels,
                  o5x5_channels,
                           pool_channels,
                  h3x3_channels=None, h5x5_channels=None,
                  phi=nn.ReLU(True)):
        super().__init__()
        self.phi = phi
        self.branch1 =
nn.Conv2d(in_channels, o1x1_channels, 1)
        self.branch2 = nn.Sequential(
            nn.Conv2d(in_channels,
h3x3_channels, 1),
            phi, nn.Conv2d(h3x3_channels,
o3x3_channels, 3, padding=1),
        )
        m5x5_channels = (h5x5_channels +
o5x5_channels) // 2
        self.branch3 = nn.Sequential(
```

Antwort

1

<pre> nn.Conv2d(in_channels, h3x3_channels, 1), phi, nn.Conv2d(h5x5_channels, m5x5_channels, 3, padding=1), phi, nn.Conv2d(m5x5_channels, o5x5_channels, 3, padding=1),) h_channels = o1x1_channels + o3x3_channels + o5x5_channels self.linear = nn.Conv2d(h_channels, in_channels, 1) def forward(self, x): s1 = self.branch1(x) s2 = self.branch2(x) s3 = self.branch3(x) s = torch.cat([s1, s2, s3], dim=1) return self.phi(x + self.linear(s))</pre>	
<pre>import torch from torch import nn class InceptionBlock(nn.Module): def __init__(self, in_channels, o1x1_channels, o3x3_channels, o5x5_channels,</pre>	Antwort 2

```
        pool_channels,
h3x3_channels=None, h5x5_channels=None,
phi=nn.ReLU(True)):
    super().__init__()
    self.phi = phi
    self.branch1 =
nn.Conv2d(in_channels, o1x1_channels, 1)
    self.branch2 = nn.Sequential(
        nn.Conv2d(in_channels,
h3x3_channels, 1),
        phi, nn.Conv2d(h3x3_channels,
o3x3_channels, 3, padding=1)
    )
    self.branch3 = nn.Sequential(
        nn.Conv2d(in_channels,
h5x5_channels, 1),
        phi, nn.Conv2d(h5x5_channels,
o5x5_channels, 3, padding=1),
        phi, nn.Conv2d(o5x5_channels,
o5x5_channels, 3, padding=1),
    )
    self.branch4 = nn.Sequential(
        nn.MaxPool2d(3, stride=1,
padding=1),
        nn.Conv2d(in_channels,
pool_channels, 1),
    )

    def forward(self, x):
```

<pre>s1 = self.branch1(x) s2 = self.branch2(x) s3 = self.branch3(x) s4 = self.branch4(x) s = torch.cat([s1, s2, s3, s4], dim=1) return self.phi(s)</pre>	
<pre>import torch from torch import nn class InceptionBlock(nn.Module): def __init__(self, in_channels, o1x1_channels, o3x3_channels, o5x5_channels, pool_channels, h3x3_channels=None, h5x5_channels=None, phi=nn.ReLU(True)): super().__init__() self.phi = phi self.branch1 = nn.Conv2d(in_channels, o1x1_channels, 1) self.branch2 = nn.Sequential(nn.Conv2d(in_channels, h3x3_channels, 1), phi, nn.Conv2d(h3x3_channels, o3x3_channels, 3, padding=1),</pre>	Antwort 3


```
        )
        self.branch3 = nn.Sequential(
            nn.Conv2d(in_channels,
h5x5_channels, 1),
            phi, nn.Conv2d(h5x5_channels,
o5x5_channels, 5, padding=2),
        )
        self.branch4 = nn.Sequential(
            nn.MaxPool2d(3, stride=1,
padding=1),
            nn.Conv2d(in_channels,
pool_channels, 1),
        )

    def forward(self, x):
        s1 = self.branch1(x)
        s2 = self.branch2(x)
        s3 = self.branch3(x)
        s4 = self.branch4(x)
        s = torch.cat([s1, s2, s3, s4],
dim=1)
        return self.phi(s)
```